GUIDED LAB B · INFRASTRUCTURE FOR BIG DATA

Infrastructure as Code: Software, Faults and Fixes

Rebuild the cluster from code, install Nginx as code, and repair a broken Vagrantfile

Student details	
Name	
Date	

Overview

> **Scenario:** At the end of Lab A you destroyed your cluster and kept only the Vagrantfile. Today you get the cluster back with one command, add software to its definition so every node arrives ready to serve, and repair a Vagrantfile that someone else broke.

You work individually on your lab PC. **Suggested time: about 55 minutes** of your second 1-hour block. Lab A must be complete.

By the end of this lab you will be able to

- Rebuild an environment from its code on any machine
- Add software installation to a Vagrantfile with a named provisioner
- Explain why a rebuilt node gets the same software with no extra steps
- Diagnose a failed build by writing a hypothesis before changing anything
- Explain which kinds of fault code review would catch, and which only testing finds

Lab settings

Use this table whenever the lab tells you to look up a value.

Setting	Value
Lab folders	**iac-vagrant-lab\labB-start** (sections 1 and 2) and **iac-vagrant-lab\labB-broken** (section 3)
Box (base image)	`ubuntu/jammy64` (Ubuntu 22.04), pre-loaded on the lab PC
Cluster network	NAT Network **VagrantNet**, 10.20.20.0/24, DHCP off
Nodes	node01 10.20.20.11, node02 10.20.20.12, node03 10.20.20.13
Web page on each node	`http://node01`, `http://node02`, `http://node03` (from inside the cluster)
Inside each VM	user **student**, password `student` (lab only)

Before you begin

1. Shut down any other VMs. Three lab nodes need about 3 GB of free memory.
2. Copy the Vagrantfile you saved at the end of Lab A into **labB-start**, replacing the one there. If you did not save yours, use the one provided: it is the 3-node cluster from Lab A.
3. Run `vagrant box list` and check that `ubuntu/jammy64` is listed.

1 Rebuild from code

Why this matters: Your cluster was destroyed at the end of Lab A, and this may not even be the same PC. If the cluster is defined as code, getting it back is one command. The build takes 6 to 8 minutes for three nodes.

1. Start the rebuild:

```
cd iac-vagrant-lab/labB-start
vagrant up
```

2. While it builds, answer the discussion question below.
3. When it finishes, check all three nodes:

```
vagrant status
vagrant ssh node01 -c "ping -c 2 node02; ping -c 2 node03"
vagrant ssh node01 -c "nproc"
```

What you found	Result
Vagrantfile used (yours from Lab A / provided)	
Time to rebuild all three nodes (minutes)	
node01 pings node02 and node03 by name? (Y/N)	
CPUs on node01 (what does this tell you about your Lab A change?)	

Discussion: To rebuild your manual cluster on a new PC, what would you have needed, and how long would it take? What did you need today?

Notes, observations, screenshots

2 Software as code: Nginx on every node

Why this matters: Building the VMs is only half the job; the software on them has to be repeatable too. A second, named provisioner installs Nginx on every node, so any node you rebuild arrives ready to serve, with no manual steps.

1. Open the Vagrantfile in **labB-start** and find this line inside the node loop: `node.vm.provision "shell", inline: PROVISION, args: [ip, HOSTS]`
2. Directly below it, still inside the `config.vm.define` block, add the **web** provisioner and save:

```
# Software as code: install Nginx and serve the hostname
node.vm.provision "web", type: "shell", inline: <<-WEB
  apt-get update -qq
  DEBIAN_FRONTEND=noninteractive apt-get install -y -qq nginx
  echo "<h1>$(hostname)</h1>" > /var/www/html/index.html
  systemctl enable --now nginx
WEB
```

3. Run only the new provisioner on every node, then fetch each page from node01:

```
vagrant provision --provision-with web
vagrant ssh node01 -c "curl -s node01; curl -s node02; curl -s node03"
```

4. Run `vagrant provision --provision-with web` a second time and read the output. What does apt do when Nginx is already installed?
5. Prove the software is part of the definition: rebuild node03 from nothing and fetch its page.

```
vagrant destroy -f node03
vagrant up node03
vagrant ssh node01 -c "curl -s node03"
```

Watch out: The VMs download Nginx from the internet through NIC1. If `apt-get` fails, test with `vagrant ssh node01 -c "ping -c 2 archive.ubuntu.com"` and tell your tutor.

What you did	Result
Output of `curl -s node02` from node01	
Second run of the web provisioner: what changed?	
Rebuilt node03 serves its page with no extra steps? (Y/N)	
Lines you added to the Vagrantfile	

Discussion: Should installing and configuring software live in the Vagrantfile, or in a separate tool such as Ansible? Give one reason for each side.

Notes, observations, screenshots

3 Break and fix

Why this matters: Code makes a mistake repeatable, so it also makes it findable. The Vagrantfile in **labB-broken** describes a 2-node cluster with **four faults**. Good troubleshooting means writing down what you think is wrong before you change anything.

1. Destroy your working cluster to free memory (you can rebuild it with `vagrant up` at any time), then try the broken one:

```
vagrant destroy -f
cd ../labB-broken
vagrant up
```

2. Work through the faults one at a time. For each one, fill in a row of the fault log, make **one** fix, then run the build again.
3. A fault in the provisioning script or in **NODES** does not need a rebuild: fix it and run `vagrant provision`.

Watch out: Do not compare this file with your own Vagrantfile until you have logged a hypothesis for every fault. If you use `diff` at the end, note that this is exactly what `git diff` gives you on real infrastructure code.

#	Symptom: what you saw	Hypothesis	Fix you made	How you confirmed it
1				
2				
3				
4				

Hints: (1) Two faults stop Vagrant before any VM starts; two only show once both nodes are running.

(2) A Ruby error's line number is where Ruby noticed the problem, not always where the problem is.

(3) Useful commands: `vagrant validate`, `vagrant box list`, `vagrant ssh node01 -c "ip -4 addr"`, `cat /etc/hosts`, `hostname -I`.

Notes, observations, screenshots

4 Final check

Why this matters: Before you leave, confirm that what you built matches what the code says. This sheet is your evidence.

Check	Expected	Actual
Section 1: nodes rebuilt from code	node01, node02, node03	
Section 2: pages served	node01 … node03	
Section 3: node01 IP on enp0s8	10.20.20.11	
Section 3: node02 IP on enp0s8	10.20.20.12	
Section 3: node01 ↔ node02 by name	Both directions work	
VMs left running after `vagrant destroy -f`	None	

Lab complete: check before you leave

- Cluster rebuilt from your Lab A Vagrantfile
- web provisioner added, every node serving its hostname, node03 rebuilt and serving with no extra steps
- Four faults logged, each with a hypothesis, fix and confirmation
- Your final Vagrantfile (with the web provisioner) saved, and all lab VMs destroyed with `vagrant destroy -f`
- Mistakes and lessons log updated (below)

Stretch (if you finish early): (1) Replace the hand-written **NODES** list with a count: `NODES = (1..NODE_COUNT).to_h { |i| ["node%02d" % i, "10.20.20.#{10 + i}"] }`

(2) Add a forwarded port to each node (guest port 80, a different host port per node) so you can open every page from your own browser.

(3) Sketch the provisioner that would turn these three web servers into the Pacemaker active/passive cluster from the slides. What has to happen on every node, and what only once?

Discussion: (1) In section 3, two nodes shared an IP. What did that look like, and how would the same mistake show up in a manual build?

(2) Which of the four faults would code review or `git diff` have caught before anyone ran `vagrant up`, and which only showed when the system ran?

(3) After Labs A and B, which steps of your manual cluster build would you now refuse to do by hand, and why?

Mistakes and lessons log: what went wrong, how you found it, what you would do differently next time.